

Estudo Comparativo de Estruturas em Árvores Avançadas: Modelagem, Análise Teórica e Avaliação Empírica

Heitor Henrique Zonho

Centro Federal de Educação Tecnológica de Minas Gerais (CEFET-MG)

Repositório: https://github.com/HeitorHenriqueZ/Trabalho_Arvores

Resumo

Este trabalho apresenta uma investigação comparativa de cinco estruturas hierárquicas avançadas de dados desenvolvidas em C++17: Trie, Árvore Patricia (modelada como Radix Tree com compressão de caminhos), Árvore Splay, Treap e KD-Tree ($K = 2$). Para cada estrutura, analisam-se as propriedades invariantes, a topologia e ocupação dos nós em memória, os algoritmos fundamentais de inserção, busca e remoção, além do rastreamento visual de seus estados dinâmicos. A análise assintótica contrapõe essas estruturas à Árvore Binária de Busca clássica e à Árvore AVL. A avaliação experimental afere o desempenho para cardinalidades $N \in \{10^4, 10^5, 5 \cdot 10^5\}$ sob distribuições uniforme, estritamente ordenada e com localidade de acesso temporal (80-20), correlacionando o consumo de nós e tempo de execução aos limites teóricos. Ao final, a Seção 7 sintetiza as conclusões fundamentadas nas questões diretas do estudo.

1 Introdução e Fundamentação Teórica

Árvores Binárias de Busca (BST) e Árvores AVL organizam chaves sob ordem total unidimensional [2]. Na BST, o caso médio de busca, inserção e remoção é $O(\log n)$; com chaves ordenadas, a topologia degenera em lista linear com custo $O(n)$. A AVL impõe a invariante de equilíbrio $BF(u) = h(u.right) - h(u.left) \in \{-1, 0, 1\}$, limitando a altura máxima a aproximadamente $1,44 \log_2(n + 2)$ e garantindo $O(\log n)$ no pior caso [5] à custa de rotações e metadados de altura por nó.

Essas estruturas de busca convencionais apresentam limitações em cenários específicos:

1. **Cadeias de caracteres (strings):** comparações de chaves de comprimento m demandam até $O(m)$ operações, elevando o custo em BST e AVL a $O(m \log n)$;
2. **Localidade temporal de referência:** padrões de acesso assimétricos ou concentrados (lei de Zipf / regra 80-20) não alteram a topologia de busca da AVL ou da BST estática;
3. **Espaço multidimensional:** não existe ordenação linear natural total para tuplas ou pontos espaciais multidimensionais $(x, y, \dots)$.

1.1 Invariantes das cinco estruturas

Trie [3]. Cada aresta corresponde a um símbolo de Σ . O caminho da raiz até o nível d fixa o prefixo de tamanho d ; um nó é terminal somente se o indicador booleano `isEndOfWord` estiver ativo.

Patricia [4]. Compactam-se caminhos unários: nós internos não terminais, salvo a raiz sentinela, têm grau ≥ 2 , e cadeias de caracteres contíguas sem bifurcação são comprimidas em uma única aresta rotulada com `prefix`. Nesta implementação, adota-se a formulação de Radix Tree / Compact Trie com rótulos de string (substrings contíguas), diferenciando-se da Patricia canônica estrita que opera com índices numéricos de desvio em nível de bits individuais (*bit-skip index*). Mantém-se, contudo, a propriedade essencial de compressão de caminhos com divisão (*split*) e fusão (*merge*) sob demanda.

Splay [7]. BST sem metadados extras. Após acesso à chave k , rotações Zig / Zig-Zig / Zig-Zag levam k (ou o último nó visitado) à raiz, preservando a ordem simétrica.

Treap [6]. Cada nó associa chave ordenada e prioridade aleatória (Max-Heap), satisfazendo $\text{prio}(u) \geq \text{prio}(u.\text{left})$ e $\text{prio}(u) \geq \text{prio}(u.\text{right})$.

KD-Tree [1]. Em profundidade d , o eixo é $\kappa = d \bmod K$. À esquerda, $v[\kappa] < u[\kappa]$; à direita, $w[\kappa] \geq u[\kappa]$.

1.2 Anatomia dos nós

Estrutura	Campos	Organização
Trie	<code>children[256]</code> , <code>isEndOfWord</code>	256 ponteiros por nó (≈ 2048 B em 64 bits)
Patricia	<code>prefix</code> , <code>children</code> , <code>flag</code>	<code>string+vector</code> ; alocação sob demanda
Splay	<code>key</code> , <code>left</code> , <code>right</code>	Sem altura/cor; ≈ 24 B/nó
Treap	campos da Splay + <code>priority</code>	24 B no ABI x86_64 verificado (usa padding)
KD-Tree	<code>coords[K]</code> , <code>left</code> , <code>right</code>	$K = 2$: duas coordenadas inteiras

Tabela 1: Anatomia dos nós (arquitetura 64 bits).

2 Projeto e Implementação das Estruturas

Implementação em ISO C++17, com cabeçalhos em `include/` e fontes em `src/`. Cópia e movimentação das classes foram desabilitadas (= `delete`) para evitar double-free. Duplicatas são rejeitadas na Splay, Treap e KD-Tree; em Trie/Patricia a reinscrição apenas reativa o marcador terminal.

2.1 Memória e build

Com o intuito de mitigar riscos de esgotamento de pilha (*stack overflow*) decorrentes de recursão profunda durante a desalocação de árvores severamente desbalanceadas em cargas elevadas ($N = 500.000$), os destrutores das árvores binárias (Splay, Treap e KD-Tree) foram implementados mediante desalocação iterativa com rotações de cauda (padrão Day-Stout-Warren), assegurando consumo auxiliar de espaço $O(1)$. Todas as estruturas implementam semântica estrita de gerenciamento de recursos (RAII), com cópias e movimentações explicitamente suprimidas (= `delete`) para impedir cópias rasas acidentais e liberação dupla (*double-free*). O projeto adota compilação limpa sob `-std=c++17 -Wall -Wextra -pedantic` no GCC, com testes automatizados em `tests/test_trees.cpp` e verificação contínua de vazamento de memória via AddressSanitizer (`make asan`). O código-fonte integral encontra-se referenciado no repositório oficial do projeto.

2.2 Operações

2.2.1 Trie

Inserção aloca filhos sob demanda e marca o terminal. Remoção desmarca e apaga nós sem filhos no retorno da recursão.

2.2.2 Patricia

Se o prefixo comum L for estritamente menor que $|u.\text{prefix}|$, ocorre *split* (Listagem 1). Na remoção, nós não terminais com um único filho são fundidos.

Listing 1: Split na Patricia

```

1 int L = commonPrefixLength(child->prefix, word);
2 if (L > 0 && L < (int)child->prefix.length()) {
3     PatriciaNode* split = new PatriciaNode(child->prefix.substr(L));
4     split->children = child->children; split->isEndOfWord = child->isEndOfWord;
5     child->prefix = child->prefix.substr(0, L); child->children = { split };
6     child->isEndOfWord = (L == (int)word.length());
7     if (!child->isEndOfWord) {
8         PatriciaNode* novo = new PatriciaNode(word.substr(L));
9         novo->isEndOfWord = true; child->children.push_back(novo);
10    }
11 }
```

2.2.3 Splay

`splay(root, key)` aplica recursivamente Zig-Zig/Zig-Zag. Na remoção, a chave sobe à raiz; as subárvores são reunidas pelo máximo da esquerda.

2.2.4 Treap

Inserção BST com rotações na volta para restaurar o Max-Heap (Algoritmo 2). Remoção rotaciona o alvo até haver no máximo um filho, que o substitui. Prioridades usam `std::mt19937`.

Listing 2: Inserção na Treap (Max-Heap)

```
1 Insert(node, key, priority):
2     if node == null: return new Node(key, priority)
3     if key == node.key: return node
4     if key < node.key:
5         node.left = Insert(node.left, key, priority)
6         if node.left.priority > node.priority:
7             node = RightRotate(node)
8     else:
9         node.right = Insert(node.right, key, priority)
10        if node.right.priority > node.priority:
11            node = LeftRotate(node)
12    return node
```

2.2.5 KD-Tree (K = 2)

Comparação em $cd = \text{depth} \bmod 2$. Remoção (Algoritmo 3): substitui pelo mínimo na dimensão cd da subárvore direita; se só houver esquerda, move-a para a direita antes de remover o substituto.

Listing 3: Remoção na KD-Tree

```
1 RemoveHelper(root, pt, depth):
2     if root == null: return null
3     cd = depth % 2
4     if root.point == pt:
5         if root.right != null:
6             min = FindMin(root.right, cd, depth + 1)
7             root.point = min.point
8             root.right = RemoveHelper(root.right, min.point, depth + 1)
9         else if root.left != null:
10            min = FindMin(root.left, cd, depth + 1)
11            root.point = min.point
12            root.right = RemoveHelper(root.left, min.point, depth + 1)
13            root.left = null
14        else:
15            delete root
16            return null
17        return root
18    if pt[cd] < root.point[cd]:
19        root.left = RemoveHelper(root.left, pt, depth + 1)
20    else:
21        root.right = RemoveHelper(root.right, pt, depth + 1)
22    return root
```

Vizinho Mais Próximo (Nearest Neighbor). A operação `nearestNeighbor` percorre a árvore de forma recursiva, mantendo o melhor candidato encontrado e podando subárvores cujo hiperplano separador está mais distante do que a melhor distância corrente. A cada nível, o algoritmo visita primeiro o lado do espaço que contém o ponto alvo, atualizando a distância mínima, e só explora o outro lado se a distância ao hiperplano ($|\text{target}[cd] - \text{node}[cd]|^2$) for inferior à melhor distância conhecida. Em baixa dimensão e sob distribuição favorável, o custo esperado é $O(\log n)$; o pior caso é $O(n)$.

Busca por Faixa (Range Search). A operação `rangeSearch` retorna todos os pontos contidos em um retângulo ortogonal $[\text{low}, \text{high}]$. Em cada nó, o algoritmo verifica se o ponto pertence à faixa e, pelo eixo discriminante cd , poda subárvores que não podem interceptar o retângulo: a subárvore esquerda é visitada apenas se $\text{low}[cd] \leq \text{node}[cd]$, e a direita apenas se $\text{high}[cd] \geq \text{node}[cd]$. Em uma árvore balanceada, a complexidade para $K = 2$ é $O(\sqrt{n} + k)$, onde k é o número de pontos reportados.

3 Demonstração e Rastreamento Visual

Para cada estrutura: (1) estado após inserções iniciais; (2) estado intermediário com o mecanismo típico; (3) estado após remoção. As quinze imagens desta seção foram geradas diretamente pelos métodos `toDot()` das cinco classes, por meio de `make diagrams`; os arquivos DOT também acompanham o código. Nós terminais de Trie e Patricia aparecem destacados, e os pequenos pontos pretos nas árvores binárias representam filhos nulos.

3.1 Trie

Figura 1: (a) “roma”; (b) “romano” reutiliza o prefixo; (c) remoção de “roma” desativa o terminal em “a”.

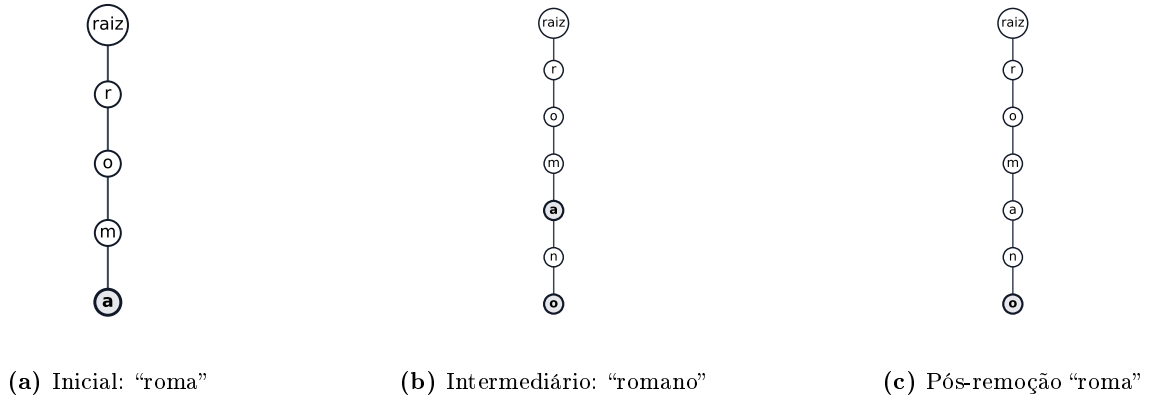


Figura 1: Trie: compartilhamento de prefixos.

3.2 Patricia

Figura 2: (a) “roma”/“romano”; (b) split com “rato”; (c) remoção de “romano”.

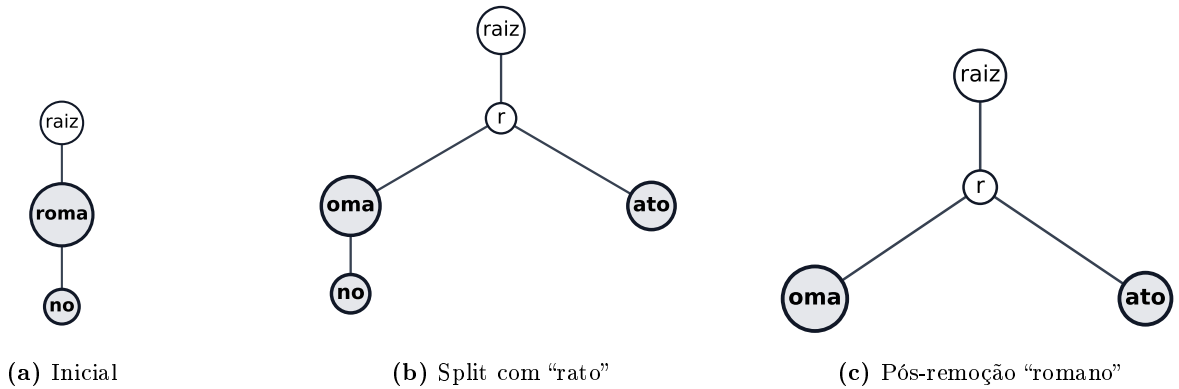


Figura 2: Patricia: compressão e split.

3.3 Splay

Figura 3: inserção ordenada 10–20–30 com splay ativo (cada inserção eleva o novo nó à raiz); busca de 10 via `search(10)`; remoção de 20. Estados verificados via `toDot()`.

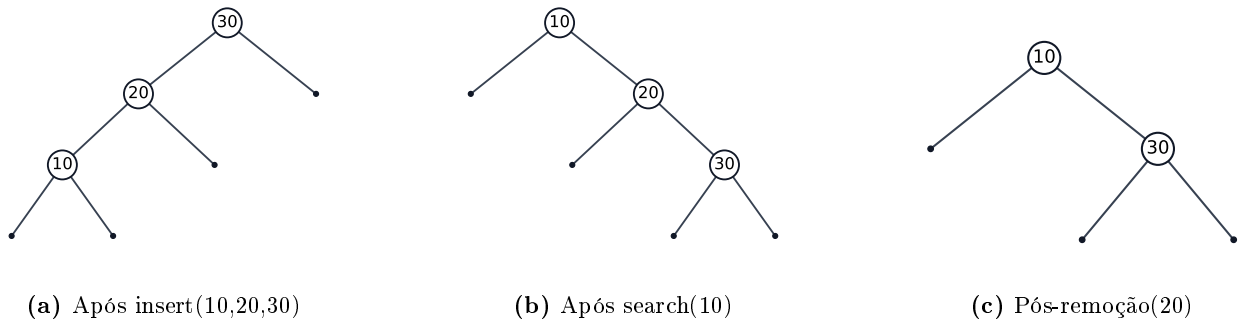


Figura 3: Splay: auto-organização por rotação.

3.4 Treap

Figura 4: (a) após inserir 50; (b) inserções de 30 e 40 com prioridades reproduzíveis a partir da semente 42, acompanhadas das rotações de Max-Heap; (c) remoção da raiz 40.

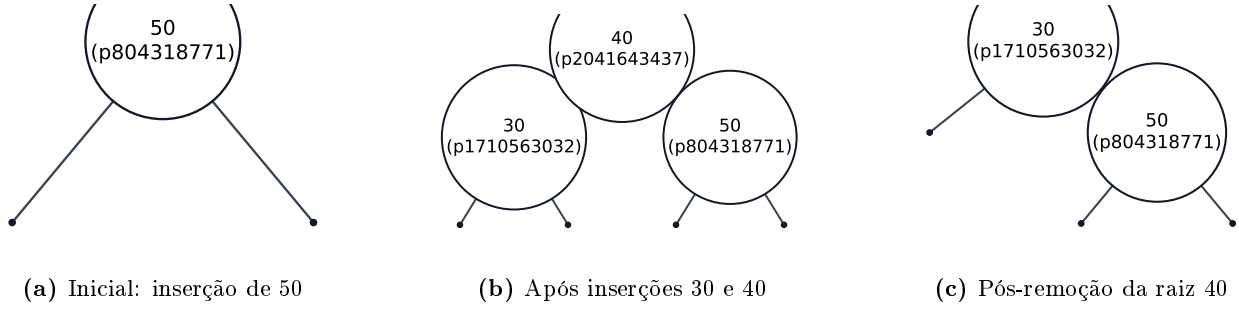


Figura 4: Treap: BST + Max-Heap.

3.5 KD-Tree (K = 2)

A Figura 5 ilustra o particionamento bidimensional com cortes ortogonais alternados e a operação de remoção estrutural: (a) estado inicial com a raiz (3,6) particionando no eixo $x = 3$ e o nó (17,15) em sua subárvore direita particionando no eixo $y = 15$; (b) estado intermediário após a inserção de (13,15), que desce pela subárvore direita da raiz ($13 \geq 3$) e pela subárvore direita de (17,15) ($15 \geq 15$), assumindo corte no eixo $x = 13$; (c) estado pós-remoção da raiz (3,6), onde o nó substituto é o mínimo na dimensão de corte x localizado na subárvore direita, correspondente a (13,15), preservando as invariantes de partição espacial.

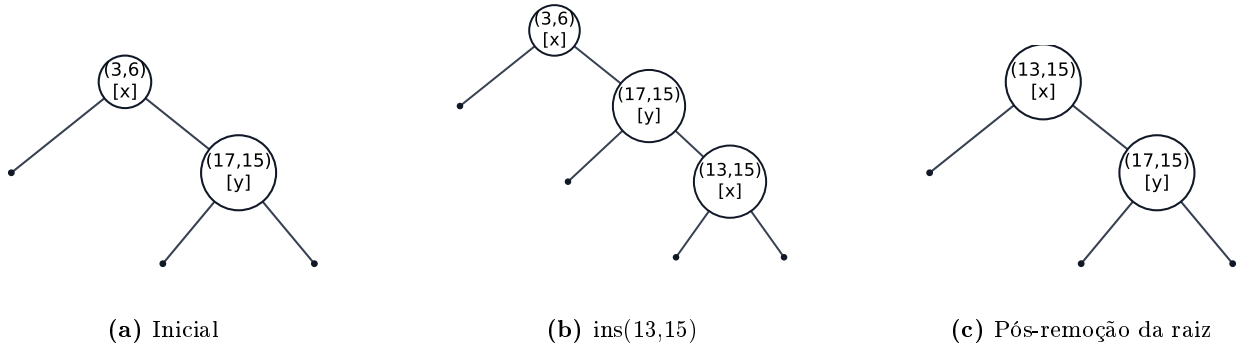


Figura 5: KD-Tree: particionamento ortogonal.

4 Análise de Complexidade e Comparação Teórica

A Tabela 2 compara as cinco estruturas com BST e AVL. Para Trie/Patricia, m é o comprimento da chave (máximo na construção) e $|\Sigma| = 256$. B/I/R: busca, inserção e remoção. A coluna melhor refere-se à busca; o caso médio depende da distribuição. Range pressupõe balanceamento e inclui a saída k .

Estrutura	Melhor	Médio B/I/R	Pior B/I/R	Construção	Espaço	Extra
Trie	$\Theta(1)$	$\Theta(m)$	$O(m)$	$O(Nm)$	$O(Nm \Sigma)$	Prefixo $O(m)$
Patricia	$\Theta(1)$	ver texto	$O(m^2)$	$O(Nm^2)$ pior	$O(Nm)$	Split local $O(m)$
Splay	$\Theta(1)$	$O(\log n)^\dagger$	$O(n)$	$O(N \log N)^\dagger$	$O(n)$	Amortizado
Treap	$\Theta(1)$	$O(\log n)$ esp.	$O(n)$	$O(N \log N)$ esp. / $O(N^2)$ pior	$O(n)$	Rotação $O(1)$
KD-Tree	$\Theta(1)$	B/I: $O(\log n)$; R: texto	$O(n)$	$O(N \log N)$ méd. / $O(N^2)$ pior	$O(n)$	Range $O(\sqrt{n} + k)$
BST	$\Theta(1)$	$O(\log n)$	$O(n)$	$O(N \log N)$ méd. / $O(N^2)$ pior	$O(n)$	—
AVL	$\Theta(1)$	$O(\log n)$	$O(\log n)$	$O(N \log N)$	$O(n)$	Altura $O(\log n)$

[†] Média amortizada (potencial de Sleator–Tarjan).

Tabela 2: Complexidade assintótica frente a BST e AVL.

Árvore Binária de Busca (BST) vs. Árvore AVL. Em uma BST estática, a topologia é inteiramente governada pela ordem de chegada das chaves. Sob sequências estritamente ordenadas, cada nova chave é inserida na extremidade da cadeia, degenerando a árvore em uma lista unária de altura $h = n - 1$. Nessa condição, busca, inserção e remoção degradam para $\Theta(n)$ e a construção total consome $\sum_{i=1}^n i = \Theta(n^2)$. A Árvore AVL contrapõe essa patologia impondo a invariante de altura $|h(u.\text{right}) - h(u.\text{left})| \leq 1$.

Uma árvore AVL de altura h com cardinalidade mínima obedece à recorrência de Fibonacci $N(h) = N(h-1) + N(h-2) + 1$, cuja solução analítica prova que $h < 1,4404 \log_2(n+2) - 0,328$. Essa cota estrita garante complexidade $O(\log n)$ no pior caso para todas as operações fundamentais, demandando no máximo duas rotações por inserção e $O(\log n)$ rotações por remoção.

Árvore Splay. A Árvore Splay dispensa metadados de balanceamento e reorganiza a árvore após os acessos. Uma operação individual pode custar $O(n)$; as rotações não reduzem a profundidade de todos os nós visitados. A garantia é amortizada: usando $r(x) = \log_2 s(x)$ e $\Phi(T) = \sum_x r(x)$, com $s(x)$ igual ao tamanho da subárvore, o lema de acesso limita o custo amortizado do splay por $3(r(t) - r(x)) + 1$, onde t é a raiz anterior. Assim, uma sequência de M operações iniciada com árvore vazia e no máximo n chaves custa $O(M \log(n+1))$. Optimalidade estática e o teorema do conjunto de trabalho são resultados adicionais sobre padrões de acesso [7].

Árvore Treap. A Treap combina a invariante simétrica da BST para as chaves com a ordenação de Max-Heap para prioridades inteiras pseudoaleatórias, geradas em um intervalo amplo no instante de inserção. No modelo teórico de prioridades independentes e distintas, sua ordenação induz a mesma distribuição de formas de uma BST construída por permutação aleatória. No código, as prioridades são inteiras e podem empatar; as comparações estritas preservam o Max-Heap não estrito, sem assegurar identidade exata com o modelo sem empates. Pela análise clássica de árvores aleatórias, a profundidade esperada é $O(\log n)$, assegurando custo esperado $O(\log n)$ para busca, inserção e remoção, embora o pior caso permaneça $O(n)$.

Trie vs. Árvore Patricia. A Trie realiza busca, inserção e remoção em $O(m)$ para alfabeto fixo; uma busca bem-sucedida percorre $\Theta(m)$ símbolos. A Radix Tree permite $O(m)$ quando evita copiar os sufixos. Nesta Patricia, porém, cada chamada a `substr` copia a parte restante da chave: em cadeias como “a”, “aa”, “aaa”, as cópias somam $m + (m-1) + \dots + 1 = O(m^2)$. Esse é o pior caso das operações implementadas; o custo médio depende dos prefixos compartilhados. A construção por N inserções tem cota $O(Nm^2)$. A Trie ocupa $O(Nm|\Sigma|)$; a Patricia, $O(Nm)$ em armazenamento permanente. Fora a raiz sentinela, nós não terminais têm ao menos dois filhos, limitando o total a $2N$ nós para $N \geq 1$ (a árvore vazia mantém a raiz). As cópias recursivas de sufixos podem consumir ainda $O(m^2)$ de espaço temporário.

KD-Tree ($K = 2$). Ao estender a partição binária para espaços multidimensionais euclidianos, a KD-Tree discrimina coordenadas alternando o hiperplano separador $\kappa = d \bmod K$ na profundidade d . A busca pontual exata em conjuntos aleatórios aproximadamente balanceados atinge tempo médio $O(\log n)$, enquanto operações avançadas como `nearestNeighbor` e `rangeSearch` exploram a poda de ramos espaciais. Para uma KD-Tree balanceada em duas dimensões, a consulta ortogonal tem cota típica $O(\sqrt{n}+k)$, onde k é a saída; sem balanceamento, busca, remoção, vizinho mais próximo e consulta por faixa podem degradar a $O(n)$. A remoção inclui `findMin`, que pode visitar $O(\sqrt{s})$ nós em uma subárvore 2D balanceada de tamanho s , pois explora ambos os ramos em eixos alternados. Assim, remover um nó próximo à raiz pode custar $O(\sqrt{n})$ mesmo com balanceamento; não se atribui custo médio logarítmico à remoção sem definir a distribuição dos alvos.

5 Metodologia Experimental e Resultados

5.1 Ambiente e método

Ubuntu 24.04 LTS (Linux x86_64, Kernel 6.6), processador Intel® Core™ i3-9100F (4 núcleos / 4 threads, 3,60 GHz), 16 GB de RAM em canal único, GPU NVIDIA GeForce GTX 1050 Ti e compilador g++ 13.3.0 (C++17, -O3). A GPU não participa dos cálculos. Tempos aferidos via `std::chrono::steady_clock`, com I/O fora da região medida e semente fixa `mt19937(42)`. Cargas sintéticas sem duplicatas: strings ASCII (5–14 caracteres), inteiros em $[1, 10^7]$ e pontos bidimensionais em $[0, 10^6]^2$. Cada valor registrado corresponde a uma execução controlada; por isso, diferenças inferiores a 1 ms devem ser interpretadas como abaixo da resolução adotada, e as conclusões privilegiam tendências de escala. Dados brutos registrados em `data/`.

5.2 Tempos medidos

Valores da Tabela 3 são cópia direta do CSV gerado nesta máquina.

Estrutura	N	Inserção (ms)	Busca (ms)	Remoção (ms)
Trie	10^4	125	1	27
Patricia	10^4	3	1	1
Splay	10^4	1	1	1
Treap	10^4	1	0	1
KD-Tree	10^4	1	0	1
Trie	10^5	1339	23	282
Patricia	10^5	46	34	32
Splay	10^5	31	24	27
Treap	10^5	26	15	21
KD-Tree	10^5	23	18	26
Trie	$5 \cdot 10^5$	5039	131	1415
Patricia	$5 \cdot 10^5$	343	333	376
Splay	$5 \cdot 10^5$	325	305	287
Treap	$5 \cdot 10^5$	273	229	235
KD-Tree	$5 \cdot 10^5$	224	179	287

Tabela 3: Tempos de benchmark_results.csv (ms).

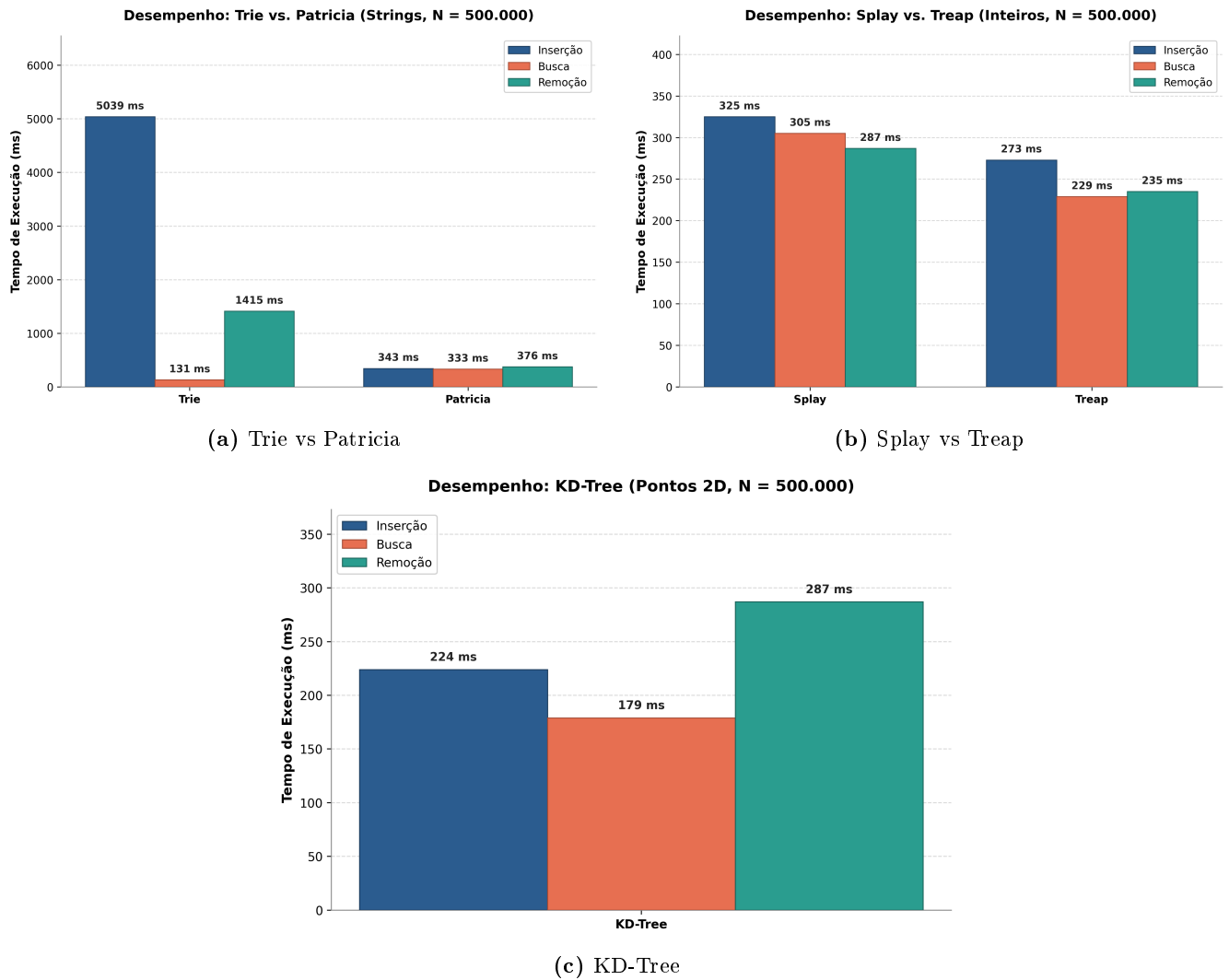


Figura 6: Desempenho para $N = 500\,000$ (mesmos valores da Tabela 3).

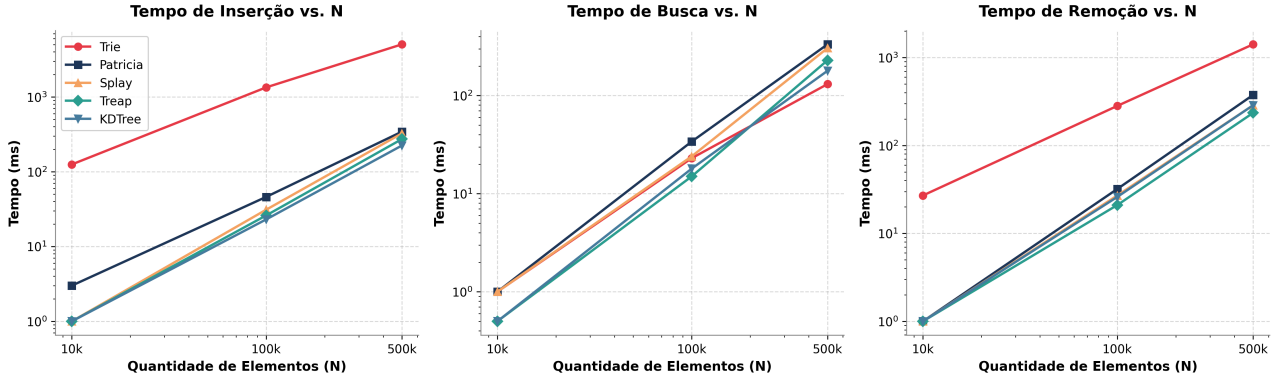


Figura 7: Escalabilidade temporal (eixos log) para os três valores de N ; medições de 0 ms são exibidas em 0,5 ms apenas para viabilizar a escala logarítmica.

5.3 Memória estrutural e outras distribuições

Estrutura	N	Nós	Razão Trie/Patricia
Trie / Patricia	10^4	73006 / 12740	5,7×
Trie / Patricia	10^5	655512 / 127398	5,1×
Trie / Patricia	$5 \cdot 10^5$	3058444 / 659604	4,6×

Tabela 4: Contagem de nós (benchmark_memory.csv).

Para aferir assimetrias estruturais e temporais, avaliaram-se inserção estritamente ordenada e uma mistura de consultas com localidade 80-20: 80% das consultas sorteiam chaves no quinto mais frequente e 20% em todo o conjunto. Esse gerador expressa localidade, mas não constitui uma distribuição de Zipf estrita. Sob inserção crescente, a Splay registra 1 ms para 10^5 chaves, pois cada novo máximo torna-se raiz. A Treap mantém altura logarítmica esperada, embora preserve pior caso linear. Na carga 80-20, a Treap foi mais rápida que a Splay nesta execução (19 ms contra 26 ms), portanto a concentração adotada não produziu vantagem para a auto-organização da Splay.

Padrão	Estrutura	N	Inserção (ms)	Busca (ms)
ordenado	Splay	10^4	0	0
ordenado	Treap	10^4	0	0
localidade 80-20	Splay	10^4	—	1
localidade 80-20	Treap	10^4	—	0
ordenado	Splay	10^5	1	6
ordenado	Treap	10^5	5	5
localidade 80-20	Splay	10^5	—	26
localidade 80-20	Treap	10^5	—	19

Tabela 5: Desempenho sob distribuições alternativas (benchmark_distributions.csv).

5.4 Confrontação com a teoria

Na inserção com $N = 5 \cdot 10^5$, Patricia (343 ms) versus Trie (5039 ms) dá fator $\approx 14,7\times$, coerente com a redução de alocações (Tabela 4). Na busca pura, a Trie foi mais rápida (131 ms vs 333 ms): comparações caractere a caractere em matriz indexada vencem varreduras de prefixos em `string`. Sob inteiros uniformes, a Treap foi mais rápida que a Splay para $N = 500.000$ (inserção 273 vs 325 ms; busca 229 vs 305 ms), mantendo a mesma ordem de grandeza esperada. A KD-Tree remove mais caro que busca (287 vs 179 ms), como previsto pelo `findMin`. A Splay não dominou o experimento 80-20 medido; a garantia amortizada não implica vantagem de tempo sobre a Treap nessa carga.

Operações espaciais da KD-Tree. Os tempos das operações espaciais implementadas, vizinho mais próximo (`nearestNeighbor`) e busca por faixa (`rangeSearch`), foram medidos com 10.000 consultas aleatórias em árvores de diversos tamanhos. A Tabela 6 apresenta os resultados.

Operação	N (árvore)	Consultas	Tempo (ms)
Nearest Neighbor	10^4	10.000	4
Nearest Neighbor	10^5	10.000	10
Nearest Neighbor	$5 \cdot 10^5$	10.000	15
Range Search	10^4	10.000	3
Range Search	10^5	10.000	13
Range Search	$5 \cdot 10^5$	10.000	81

Tabela 6: Desempenho das operações espaciais da KD-Tree.

O crescimento sublinear observado no tempo de NN ($4\text{ ms} \rightarrow 10\text{ ms} \rightarrow 15\text{ ms}$) para 10.000 consultas fixas é compatível com a poda espacial, sem constituir prova isolada da complexidade. As faixas têm lado 10.000 em um domínio de lado 10^6 : cobrem 1% de cada eixo e 0,01% da área. O aumento de 3 ms para 81 ms reflete tanto a travessia quanto o crescimento de k com N .

6 Aplicações, Análise Crítica e Discussão

As propriedades estruturais e operacionais observadas ao longo da investigação definem domínios preferenciais de aplicação prática para cada uma das estruturas analisadas:

- **Trie:** Projetada para sistemas de preenchimento automático (*autocomplete*), corretores ortográficos, dicionários de busca textual e analisadores léxicos em compiladores. A capacidade de realizar buscas por prefixo em tempo estritamente proporcional ao comprimento da chave ($O(m)$), independentemente do número de registros armazenados, sobrepuja a dependência logarítmica de árvores binárias convencionais.
- **Patricia (Radix Tree):** Amplamente empregada no encaminhamento de pacotes em redes de computadores, particularmente no algoritmo de casamento de prefixo mais longo (*longest prefix match* em tabelas de roteamento BGP/CIDR), índices invertidos em motores de busca e estruturas de dados de compressão textual. A compactação de caminhos atenua significativamente o consumo de memória frente à Trie comum ao eliminar nós intermediários unários.
- **Splay:** Ideal para implementação de memórias *cache* de software (como tabelas de tradução de endereços virtuais, caches DNS e registros de sessões web) e em cenários caracterizados por forte localidade temporal ou distribuições de acesso enviesadas (distribuição de Zipf). As chaves consultadas com frequência convergem naturalmente para o topo da hierarquia, proporcionando tempos de acesso ultra-rápidos sem sobrecarga de metadados estáticos de balanceamento.
- **Treap:** Estrutura indicada para tabelas de símbolos de uso geral, conjuntos dinâmicos ordenados com alta taxa de mutação e árvores de busca persistentes ou com suporte a operações intervalares (*split* e *merge*). A aleatorização das prioridades torna a altura logarítmica em esperança, independentemente da ordem de entrada, mas não elimina o pior caso linear.
- **KD-Tree:** Padrão fundamental em Sistemas de Informação Geográfica (GIS), algoritmos de agrupamento e vizinhos mais próximos (k -NN) em aprendizado de máquina, detecção de colisões espaciais e renderização. A alternância dos eixos particiona o espaço; o balanceamento depende da ordem de inserção ou de uma construção por medianas. Nesta implementação, `nearestNeighbor` e `rangeSearch` demonstram a poda espacial.

6.1 Dificuldades de implementação e reflexão crítica

Durante a fase de desenvolvimento, a implementação da remoção na KD-Tree representou o maior desafio algorítmico. Quando o nó a ser excluído não possui filho direito, mas possui subárvore esquerda não nula, a substituição pelo predecessor imediato habitual de uma BST unidimensional viola a invariante espacial dos eixos subsequentes. A solução exigiu localizar o nó mínimo ao longo da dimensão de corte corrente na subárvore esquerda via `findMin`, atribuir suas coordenadas ao nó corrente e migrar recursivamente a subárvore esquerda para o ramo direito (`root.right = removeHelper(root.left, min.point, depth+1)`), anulando o apontador esquerdo. Além da remoção, a implementação das operações espaciais

Estrutura	Vantagens Observadas	Limitações e Riscos Técnicos
Trie	Busca por prefixo em tempo $O(m)$; indexação direta em vetor estático de ponteiros sem colisões.	Alto consumo de memória (256 ponteiros: 2048 B; nó: 2056 B no ABI verificado); ineficiente para chaves esparsas.
Patricia	Economia expressiva de nós via compressão de caminhos (fator até $5,7\times$); inserção acelerada em grandes volumes.	Lógica elaborada de cisão (<i>split</i>) e fusão (<i>merge</i>); custo de busca ligeiramente superior devido a operações de substring.
Splay	Ausência de metadados de balanceamento; auto-organização dinâmica favorável a padrões de acesso com localidade.	Pior caso individual $O(n)$; operações de busca provocam escritas em memória (mutações topológicas), inviabilizando leituras puras concorrentes.
Treap	Complexidade $O(\log n)$ esperada com balanceamento probabilístico robusto; algoritmos de rotação concisos.	Custo de geração pseudoaleatória de prioridades; campo adicional de 4 bytes, absorvido pelo padding no ABI verificado.
KD-Tree	Busca exata, vizinho mais próximo (nearestNeighbor) e busca por faixa (rangeSearch) em espaço ortogonal com poda eficiente.	Remoção complexa e custosa (<i>findMin</i> multidimensional); degradação de desempenho em altas dimensões (maldição da dimensionalidade).

Tabela 7: Síntese comparativa de vantagens, limitações e compromissos de projeto.

nearestNeighbor e **rangeSearch** exigiu a construção cuidadosa da lógica de poda: no vizinho mais próximo, a condição $\text{diff}^2 < \text{bestDist}$ determina se o lado oposto do hiperplano deve ser explorado; na busca por faixa, a interseção do retângulo com o semi-espaço do nó governa a descida.

Na Árvore Patricia, o gerenciamento de substrings durante o *split* de nós intermediários e a subsequente fusão (*merge*) após remoções exigiram rigor extremo no cálculo de comprimentos comuns (**commonPrefixLength**), evitando a propagação de nós órfãos ou duplicatas lógicas. Já na Árvore Splay, o correto encadeamento das rotações compostas Zig-Zig e Zag-Zag, em que o nó avô deve obrigatoriamente ser rotacionado antes do nó pai, revelou-se determinante para satisfazer o potencial amortizado de Sleator e Tarjan.

Como melhorias futuras, sugerem-se: (i) representação adaptativa de filhos na Trie com tabelas hash esparsas ou arrays comprimidos para alfabetos arbitrários; (ii) construção balanceada estática da KD-Tree baseada em cálculo de medianas no tempo $O(N \log N)$; (iii) extensão da Treap para suportar operações implícitas de concatenação e partição de sequências; e (iv) implementação de k -NN (k vizinhos) com fila de prioridade na KD-Tree.

7 Conclusão

Este trabalho consolidou a modelagem formal, implementação e experimentação empírica de cinco famílias de árvores computacionais avançadas. A síntese das constatações teóricas e práticas é articulada nas respostas às questões diretivas a seguir:

1. Principais diferenças estruturais entre as abordagens. A Trie e a Árvore Patricia organizam dados como árvores digitais sobre sequências alfanuméricas, com ramificações determinadas pelos símbolos constituintes da chave. A Patricia diferencia-se por colapsar nós unários intermediários em arestas compostas, reduzindo drasticamente a quantidade de nós alocados. Em contraste, a Splay e a Treap operam sobre chaves escalares sob a invariante de ordem simétrica de Árvores Binárias de Busca, divergindo na disciplina de balanceamento: a Splay adapta-se deterministicamente a cada acesso via rotações locais que aproximam nós consultados da raiz, enquanto a Treap recorre a prioridades inteiras pseudoaleatórias e rotações de Max-Heap para assegurar altura logarítmica esperada sem depender da ordem das chaves. Por fim, a KD-Tree estende a partição binária para espaços euclidianos K -dimensionais alternando ciclicamente o eixo ortogonal discriminante a cada nível da hierarquia.

2. Desempenho empírico por classe de operação e conjunto de dados. Na manipulação de strings em larga escala ($N = 500.000$), a Árvore Patricia registrou inserção de 343 ms contra 5039 ms da Trie e remoção de 376 ms contra 1415 ms, fruto da economia de alocações. A Trie foi mais rápida na busca exata (131 ms contra 333 ms). No domínio de inteiros uniformes, a Treap superou a Splay nas três operações medidas. Sob inserção ordenada, a Splay assimilou 100.000 chaves em 1 ms. Na KD-Tree, meio

milhão de pontos produziram 179 ms em busca exata e 224 ms em inserção; 10.000 consultas de vizinho mais próximo consumiram 15 ms, e 10.000 faixas cobrindo 0,01% da área levaram 81 ms.

3. Coerência entre a teoria de complexidade e os dados experimentais. Os dados foram coerentes com as tendências teóricas, mas não bastam para identificar expoentes assintóticos com precisão. Trie e Patricia processaram N chaves de comprimento limitado, de modo que o tempo total cresceu aproximadamente com o volume de entrada; a contagem de nós evidenciou a economia estrutural da Patricia. Splay, Treap e KD-Tree apresentaram crescimento compatível com casos médios sublineares por operação, sem excluir seus piores casos lineares. Na carga de localidade 80-20, a Treap registrou 19 ms contra 26 ms da Splay, mostrando que a concentração adotada não foi suficiente para produzir vantagem mensurável da auto-organização.

4. Dificuldade de desenvolvimento e aspectos críticos de projeto. A KD-Tree e a Árvore Patricia demandaram o maior esforço de implementação. Na KD-Tree, a rotina de remoção impôs o tratamento rigoroso da alternância dimensional e a substituição estrutural pelo mínimo da subárvore esquerda em caso de filho direito nulo, sob pena de violar a invariante geométrica dos eixos subsequentes. Na Patricia, a gerência de cortes e junções de prefixos com atualização precisa de nós terminais exigiu bateria exaustiva de testes para prevenir corrupção topológica. Em contrapartida, a Treap destacou-se pela menor complexidade de código: combinando inserção clássica de BST com rotações locais de Max-Heap no retorno da recursão, assegurou garantias assintóticas robustas com implementação compacta.

5. Diretrizes de escolha para cenários práticos. Recomenda-se a Patricia para dicionários volumosos com alta taxa de compartilhamento de prefixos e sistemas de encaminhamento de pacotes sob restrição de memória; a Trie para vocabulários com alfabetos restritos onde a latência mínima de busca é crítica; a Splay para camadas de cache de software, tradução de endereços e cargas com forte viés de acesso temporal; a Treap para coleções dinâmicas de propósito geral que exijam proteção contra entradas adversariais ordenadas sem a sobrecarga de manutenção de árvores Rubro-Negras; e a KD-Tree para indexação espacial e consultas de proximidade euclidiana em dimensões moderadas ($K \leq 10$).

6. Síntese comparativa de vantagens e limitações. Conclui-se que nenhuma estrutura domina universalmente todos os cenários. A seleção adequada decorre de compromissos técnicos entre ocupação de memória física, custo computacional de rebalanceamento, complexidade de implementação e as distribuições estatísticas das entradas. A compreensão aprofundada das relações entre topologia, análise assintótica e custos reais de arquitetura constitui pré-requisito indispensável para a engenharia de software de alto desempenho.

Disponibilidade de Código e Reprodutibilidade

O código-fonte integral, testes automatizados, rotinas de compilação e dados experimentais encontram-se disponíveis publicamente no GitHub (https://github.com/HeitorHenriqueZ/Trabalho_Arvores), suportando: `make` (compilação com `-std=c++17 -Wall -Wextra -pedantic`), `make test` e `make asan` (testes de corretude e validação via AddressSanitizer), `make run` (execução de experimentos com persistência em `data/`) e scripts de análise em `tools/`.

Referências

- [1] BENTLEY, Jon Louis. Multidimensional binary search trees used for associative searching. *Communications of the ACM*, v. 18, n. 9, p. 509–517, 1975.
- [2] CORMEN, Thomas H. et al. *Introduction to Algorithms*. 3. ed. Cambridge: MIT Press, 2009.
- [3] FREDKIN, Edward. Trie memory. *Communications of the ACM*, v. 3, n. 9, p. 490–499, 1960.
- [4] MORRISON, Donald R. PATRICIA. *Journal of the ACM*, v. 15, n. 4, p. 514–534, 1968.
- [5] SEDGEWICK, Robert; WAYNE, Kevin. *Algorithms*. 4. ed. Boston: Addison-Wesley, 2011.
- [6] SEIDEL, Raimund; ARAGON, Cecilia R. Randomized search trees. *Algorithmica*, v. 16, n. 4, p. 464–497, 1996.
- [7] SLEATOR, Daniel D.; TARJAN, Robert E. Self-adjusting binary search trees. *Journal of the ACM*, v. 32, n. 3, p. 652–686, 1985.

- [8] ZONHO, Heitor Henrique. *Trabalho de Árvores Avançadas: Implementação e Avaliação Experimental*. Repositório no GitHub, 2026. Disponível em: https://github.com/HeitorHenriqueZ/Trabalho_Arvores.